

Zajęcia P1. Analizator leksykalny dla uproszczonego języka Ada

1 Cel ćwiczeń

Celem ćwiczeń jest stworzenie prostego analizatora leksykalnego dla bardzo uproszczonej wersji języka programowania Ada. Zadaniem tworzonego analizatora jest:

- rozpoznawanie elementów języka Ada i określanie ich wartości
- usuwanie białych znaków i komentarzy
- wykrywanie wskazanych dyrektyw
- wykrywanie błędów leksykalnych

2 Czynności wstępne

Po włączeniu komputera należy wybrać system operacyjny Linux i zalogować się jako użytkownik **student**. Należy otworzyć okno konsoli (np. nacisnąć **Alt-F2** i napisać **xterm**), utworzyć własny podkatalog za pomocą polecenia **mkdir nazwisko_użytkownika**, i podkatalog dla bieżącego ćwiczenia. Ze strony przedmiotu na platformie Moodle dla tematu *Analiza leksykalna* należy pobrać pliki. Znajdują się tam następujące pliki:

- **p1a.pdf** — instrukcja (właśnie czytany plik)
- **Makefile** — potrzebny do kompilacji za pomocą polecenia **make**
- **ada.l** — szkielet analizatora leksykalnego, który należy uzupełnić; należy zwrócić uwagę na definicję funkcji **process_token()**, którą należy wykorzystać
- **ada.y** — analizator składniowy, którego jedynymi zadaniami jest deklaracja rozpoznawanych elementów końcowych oraz wywołanie analizatora leksykalnego
- **test1.adb** — program testowy poprawny w danej gramatyce
- **test2.adb** — program testowy niepoprawny

Po zakończeniu pracy wskazane jest usunięcie utworzonego katalogu wraz z zawartością.

3 Zadania do wykonania

Należy uzupełnić dostarczony szkielet analizatora leksykalnego i pokazać, że działa poprawnie testując do na dostarczonych programach testowych. Analizator powinien wypisywać informacje o rozpoznanych symbolach końcowych w trzech kolumnach:

1. dopasowany tekst
2. rozpoznany symbol
3. wartość symbolu (tylko w sytuacji, gdy symbol rzeczywiście ma wartość)

Do wypisywania tych informacji służy zawarta w dostarczonym szkiecie analizatora funkcja **process_token()**. Funkcja zwraca rozpoznany symbol, dlatego w działaniu dla reguły rozpoznającej symbol powinna znaleźć się instrukcja **return process_token(. . .)** z odpowiednimi parametrami funkcji.

Dostarczony kod należy uzupełnić o następujące elementy:

- A. wypisanie własnego imienia i nazwiska (w pliku dla programu **bison**)
- B. wykrywanie słów kluczowych zdefiniowanych w pliku źródłowym dla programu **bison**
- C. usuwanie białych znaków
- D. usuwanie komentarzy jednowierszowych
- E. wykrywanie operatorów wieloznakowych (**<=**, **++**, **...**) występujących w programach testowych

- F. wykrywanie identyfikatorów
- G. wykrywanie liczb całkowitych
- H. wykrywanie liczb rzeczywistych
- I. wykrywanie stałych tekstowych (napisów) bez użycia mechanizmu warunków początkowych
- J. wykrywanie symboli końcowych jednoznakowych: operatorów, interpunkcji
- K. wykrywanie stałych znakowych
- L. wykrywanie napisów z użyciem warunków początkowych
- M. wykrywanie komentarzy z użyciem warunków początkowych (tak, to jest sztuczne)
- N. wykrywanie niezamkniętych napisów z użyciem mechanizmu warunków początkowych (koniec wiersza w napisie)
- O. wykrywanie napisów zawierających cudzysłowy (zapisywane jako dwa cudzysłowy obok siebie) z użyciem mechanizmu warunków początkowych

4 Ocena

Za każdy element można dostać 1 punkt, czyli razem 15 punktów. Jeżeli ktoś nie zdąży zrobić wszystkich elementów na zajęciach, istnieje możliwość dokończenia analizatora w domu, ale **tylko elementów od K do O** i za każdy element **w domu przysługuje tylko połowa punktów**. Plik wykonany na zajęciach umieścić **na zajęciach** na platformie Moodle. UWAGA: **Analizator leksykalny potrzebny będzie na następnych zajęciach**.

5 Przypomnienie warunków początkowych

- Warunek początkowy aktywny na początku programu: INITIAL
- Deklaracja (w pierwszej części): %x warunek1, warunek2, . . .
- Dopasowanie w określonym stanie:


```
<war1> re1      działanie1;
<war1,war2,INITIAL>re2 działanie2;
<*>re3         działanie3
```
- zmiana warunku początkowego: BEGIN warunek4
- bieżący warunek początkowy: YY_START
- sprawdzenie bieżącego warunku po odczytaniu wszystkich danych wejściowych: w funkcji yywrap, którą należy zdefiniować i która musi zwrócić wartość 1

6 Dane testowe — plik test1.adb

```

1 — Ten program drukuje znaki i ich kody ASCII
2 — Kompilacja: gnatmake listascii
3
4 with Ada.Text_IO; use Ada.Text_IO;
5 with Ada.Integer_Text_IO;
6
7 procedure ListAscii is
8   FromASCII : constant Integer := 32;
9   ToASCII   : constant Integer := 127;
10  Uc : Character := ' ';           — zmienna
11  Fl : Float;
12  T  : array(1..10) of Float;
13  I  : Integer;
14  D  : record
15     Dzień, Miesiąc : Integer;
```

```

16     Rok : Integer;
17     end record;
18 begin
19     New_Line;
20     Put_Line("Kody ASCII:"); Put_Line("=====");
21     for I in Integer range FromASCII .. ToASCII loop
22         Ada.Integer_Text_IO.Put(I);    — procedura z Ada.Integer_Text_IO
23         Put(' ');    — to jest procedura z Ada.Text_IO widoczna z powodu ,,use''
24         Put(Uc);
25         Uc := Character'Succ(Uc);    — kolejny znak
26         New_Line;
27     end loop;
28     — tylko test
29     Fl := 1.1 * 0.1 + 1.0e-2;    — w rzeczywistych po kropce i przed zawsze cyfra
30     T(0) := 0;
31     for I in 1 .. 10 loop
32         T(I) := T(I) * I * I;
33     end loop;
34     D.Dzien := 1;
35     D.Miesiac := D. Dzien * 10;
36     D.Rok := 2018;
37 end ListAscii;

```

7 Dane testowe — plik test2.adb

```

1 — Ten program drukuje znaki i ich kody ASCII
2 — Kompilacja: gnatmake listascii
3
4 with Ada.Text_IO; use Ada.Text_IO;
5 with Ada.Integer_Text_IO;
6
7 procedure ListAscii is
8     FromASCII : constant Integer := 32;
9     ToASCII : constant Integer := 127;
10    Uc : Character := ' ';    — zmienna
11    Fl : Float;
12    T : array(1..10) of Float;
13    I : Integer;
14    D : record
15        Dzien, Miesiac : Integer;
16        Rok : Integer;
17    end record;
18 begin
19     New_Line;
20     Put_Line("Kody ASCII:"); Put_Line("=====");
21     for I in Integer range FromASCII .. ToASCII loop
22         Ada.Integer_Text_IO.Put(I);    — procedura z Ada.Integer_Text_IO
23         Put(' ');    — to jest procedura z Ada.Text_IO widoczna z powodu ,,use''
24         Put(Uc);
25         Uc := Character'Succ(Uc);    — kolejny znak
26         New_Line;
27     end loop;
28     — tylko test
29     Fl := 1.1 * 0.1 + 1.0e-2;    — w rzeczywistych po kropce i przed zawsze cyfra
30     T(0) := 0;
31     for I in 1 .. 10 loop
32         T(I) := T(I) * I * I;
33     end loop;
34     D.Dzien := 1;
35     D.Miesiac := D. Dzien * 10;
36     D.Rok := 2018;
37 end ListAscii;

```

8 Wyjście analizatora leksykalnego dla test1.adb

Imie i Nazwisko	Typ tokena	Wartosc tokena znakowo
yytext		
with	KW.WITH	
Ada	IDENT	Ada
.	.	
Text_IO	IDENT	Text_IO
;	;	
use	KW.USE	
Ada	IDENT	Ada
.	.	
Text_IO	IDENT	Text_IO
;	;	
with	KW.WITH	
Ada	IDENT	Ada
.	.	
Integer_Text_IO	IDENT	Integer_Text_IO
;	;	
procedure	KW.PROCEDURE	
ListAscii	IDENT	ListAscii
is	KW.IS	
FromASCII	IDENT	FromASCII
:	:	
constant	KW.CONSTANT	
Integer	IDENT	Integer
:=	ASSIGN	
32	INTEGER.CONST	32
;	;	
ToASCII	IDENT	ToASCII
:	:	
constant	KW.CONSTANT	
Integer	IDENT	Integer
:=	ASSIGN	
127	INTEGER.CONST	127
;	;	
Uc	IDENT	Uc
:	:	
Character	IDENT	Character
:=	ASSIGN	
' '	CHARACTER.CONST	' '
;	;	
F1	IDENT	F1
:	:	
Float	IDENT	Float
;	;	
T	IDENT	T
:	:	
array	KW.ARRAY	
(	(	
1	INTEGER.CONST	1
..	RANGE	
10	INTEGER.CONST	10
)	)	
of	KW.OF	
Float	IDENT	Float
;	;	
I	IDENT	I
:	:	
Integer	IDENT	Integer
;	;	
D	IDENT	D
:	:	
record	KW.RECORD	
Dzien	IDENT	Dzien

65	,	,	
66	Miesiac	IDENT	Miesiac
67	:	:	
68	Integer	IDENT	Integer
69	;	;	
70	Rok	IDENT	Rok
71	:	:	
72	Integer	IDENT	Integer
73	;	;	
74	end	KW.END	
75	record	KW.RECORD	
76	;	;	
77	begin	KW.BEGIN	
78	New_Line	IDENT	New_Line
79	;	;	
80	Put_Line	IDENT	Put_Line
81	(	(	
82	"Kody ASCII:"	STRING.CONST	"Kody ASCII:"
83	)	)	
84	;	;	
85	Put_Line	IDENT	Put_Line
86	(	(	
87	"=====	STRING.CONST	"=====
88	" "	STRING.CONST	" "
89	)	)	
90	;	;	
91	for	KW.FOR	
92	I	IDENT	I
93	in	KW.IN	
94	Integer	IDENT	Integer
95	range	KW.RANGE	
96	FromASCII	IDENT	FromASCII
97	..	RANGE	
98	ToASCII	IDENT	ToASCII
99	loop	KW.LOOP	
100	Ada	IDENT	Ada
101	.	.	
102	Integer_Text_IO	IDENT	Integer_Text_IO
103	.	.	
104	Put	IDENT	Put
105	(	(	
106	I	IDENT	I
107	)	)	
108	;	;	
109	Put	IDENT	Put
110	(	(	
111	' '	CHARACTER.CONST	' '
112	)	)	
113	;	;	
114	Put	IDENT	Put
115	(	(	
116	Uc	IDENT	Uc
117	)	)	
118	;	;	
119	Uc	IDENT	Uc
120	:=	ASSIGN	
121	Character	IDENT	Character
122	,	,	
123	Succ	IDENT	Succ
124	(	(	
125	Uc	IDENT	Uc
126	)	)	
127	;	;	
128	New_Line	IDENT	New_Line
129	;	;	
130	end	KW.END	

131	loop	KWLOOP	
132	;	;	
133	F1	IDENT	F1
134	:=	ASSIGN	
135	1.1	FLOAT.CONST	1.1
136	*	*	
137	0.1	FLOAT.CONST	0.1
138	+	+	
139	1.0e-2	FLOAT.CONST	1.0e-2
140	;	;	
141	T	IDENT	T
142	(	(	
143	0	INTEGER.CONST	0
144	)	)	
145	:=	ASSIGN	
146	0	INTEGER.CONST	0
147	;	;	
148	for	KWFOR	
149	I	IDENT	I
150	in	KW.IN	
151	1	INTEGER.CONST	1
152	..	RANGE	
153	10	INTEGER.CONST	10
154	loop	KWLOOP	
155	T	IDENT	T
156	(	(	
157	I	IDENT	I
158	)	)	
159	:=	ASSIGN	
160	T	IDENT	T
161	(	(	
162	I	IDENT	I
163	)	)	
164	*	*	
165	I	IDENT	I
166	*	*	
167	I	IDENT	I
168	;	;	
169	end	KW.END	
170	loop	KWLOOP	
171	;	;	
172	D	IDENT	D
173	.	.	
174	Dzien	IDENT	Dzien
175	:=	ASSIGN	
176	1	INTEGER.CONST	1
177	;	;	
178	D	IDENT	D
179	.	.	
180	Miesiac	IDENT	Miesiac
181	:=	ASSIGN	
182	D	IDENT	D
183	.	.	
184	Dzien	IDENT	Dzien
185	*	*	
186	10	INTEGER.CONST	10
187	;	;	
188	D	IDENT	D
189	.	.	
190	Rok	IDENT	Rok
191	:=	ASSIGN	
192	2018	INTEGER.CONST	2018
193	;	;	
194	end	KW.END	
195	ListAscii	IDENT	ListAscii
196	;	;	

9 Wyjście analizatora leksykalnego dla test2.adb

Imie i Nazwisko	Typ tokena	Wartosc tokena znakowo
yytext		
with	KW.WITH	
Ada	IDENT	Ada
.	.	
Text_IO	IDENT	Text_IO
;	;	
use	KW.USE	
Ada	IDENT	Ada
.	.	
Text_IO	IDENT	Text_IO
;	;	
with	KW.WITH	
Ada	IDENT	Ada
.	.	
Integer_Text_IO	IDENT	Integer_Text_IO
;	;	
procedure	KW.PROCEDURE	
ListAscii	IDENT	ListAscii
is	KW.IS	
FromASCII	IDENT	FromASCII
:	:	
constant	KW.CONSTANT	
Integer	IDENT	Integer
:=	ASSIGN	
32	INTEGER.CONST	32
;	;	
ToASCII	IDENT	ToASCII
:	:	
constant	KW.CONSTANT	
Integer	IDENT	Integer
:=	ASSIGN	
127	INTEGER.CONST	127
;	;	
Uc	IDENT	Uc
:	:	
Character	IDENT	Character
:=	ASSIGN	
' ,	CHARACTER.CONST	' ,
;	;	
F1	IDENT	F1
:	:	
Float	IDENT	Float
;	;	
T	IDENT	T
:	:	
array	KW.ARRAY	
(	(	
1	INTEGER.CONST	1
..	RANGE	
10	INTEGER.CONST	10
)	)	
of	KW.OF	
Float	IDENT	Float
;	;	
I	IDENT	I
:	:	
Integer	IDENT	Integer
;	;	

61	D	IDENT	D
62	:	:	
63	record	KW.RECORD	
64	Dzien	IDENT	Dzien
65	,	,	
66	Miesiac	IDENT	Miesiac
67	:	:	
68	Integer	IDENT	Integer
69	;	;	
70	Rok	IDENT	Rok
71	:	:	
72	Integer	IDENT	Integer
73	;	;	
74	end	KW.END	
75	record	KW.RECORD	
76	;	;	
77	begin	KW.BEGIN	
78	New_Line	IDENT	New_Line
79	;	;	
80	Put_Line	IDENT	Put_Line
81	(	(	
82	"Kody ASCII:"	STRING.CONST	"Kody ASCII:"
83	)	)	
84	;	;	
85	Put_Line	IDENT	Put_Line
86	(	(	
87	"_____"	STRING.CONST	"_____"
88	New line in a string constant!		
89	for	KW.FOR	
90	I	IDENT	I
91	in	KW.IN	
92	Integer	IDENT	Integer
93	range	KW.RANGE	
94	FromASCII	IDENT	FromASCII
95	..	RANGE	
96	ToASCII	IDENT	ToASCII
97	loop	KW.LOOP	
98	Ada	IDENT	Ada
99	.	.	
100	Integer_Text_IO	IDENT	Integer_Text_IO
101	.	.	
102	Put	IDENT	Put
103	(	(	
104	I	IDENT	I
105	)	)	
106	;	;	
107	Put	IDENT	Put
108	(	(	
109	' ,	CHARACTER.CONST	' ,
110	)	)	
111	;	;	
112	Put	IDENT	Put
113	(	(	
114	Uc	IDENT	Uc
115	)	)	
116	;	;	
117	Uc	IDENT	Uc
118	:=	ASSIGN	
119	Character	IDENT	Character
120	,	,	
121	Succ	IDENT	Succ
122	(	(	
123	Uc	IDENT	Uc
124	)	)	
125	;	;	
126	New_Line	IDENT	New_Line

127	;	;	
128	end	KW.END	
129	loop	KW.LOOP	
130	;	;	
131	F1	IDENT	F1
132	:=	ASSIGN	
133	1.1	FLOAT.CONST	1.1
134	*	*	
135	0.1	FLOAT.CONST	0.1
136	+	+	
137	1.0e-2	FLOAT.CONST	1.0e-2
138	;	;	
139	T	IDENT	T
140	(	(	
141	0	INTEGER.CONST	0
142	)	)	
143	:=	ASSIGN	
144	0	INTEGER.CONST	0
145	;	;	
146	for	KW.FOR	
147	I	IDENT	I
148	in	KW.IN	
149	1	INTEGER.CONST	1
150	..	RANGE	
151	10	INTEGER.CONST	10
152	loop	KW.LOOP	
153	T	IDENT	T
154	(	(	
155	I	IDENT	I
156	)	)	
157	:=	ASSIGN	
158	T	IDENT	T
159	(	(	
160	I	IDENT	I
161	)	)	
162	*	*	
163	I	IDENT	I
164	*	*	
165	I	IDENT	I
166	;	;	
167	end	KW.END	
168	loop	KW.LOOP	
169	;	;	
170	D	IDENT	D
171	.	.	
172	Dzien	IDENT	Dzien
173	:=	ASSIGN	
174	1	INTEGER.CONST	1
175	;	;	
176	D	IDENT	D
177	.	.	
178	Miesiac	IDENT	Miesiac
179	:=	ASSIGN	
180	D	IDENT	D
181	.	.	
182	Dzien	IDENT	Dzien
183	*	*	
184	10	INTEGER.CONST	10
185	;	;	
186	D	IDENT	D
187	.	.	
188	Rok	IDENT	Rok
189	:=	ASSIGN	
190	2018	INTEGER.CONST	2018
191	;	;	
192	end	KW.END	

193	ListAscii	IDENT	ListAscii
194	;	;	